Latest updates: https://dl.acm.org/doi/10.1145/3689031.3696062

RESEARCH-ARTICLE

Deft: A Scalable Tree Index for Disaggregated Memory

JING WANG, Tsinghua University, Beijing, China

QING WANG, Tsinghua University, Beijing, China

YUHAO ZHANG, Tsinghua University, Beijing, China

JIWU SHU, Tsinghua University, Beijing, China

Open Access Support provided by:

Tsinghua University

PDF Download
3689031.3696062.pdf
17 January 2026
Total Citations: 1
Total Downloads: 2084

Published: 30 March 2025

Citation in BibTeX format

EuroSys '25: Twentieth European Conference on Computer Systems
March 30 - April 3, 2025
Rotterdam, Netherlands

Conference Sponsors:
SIGOPS

EuroSys '25: Proceedings of the Twentieth European Conference on Computer Systems (March 2025)
https://doi.org/10.1145/3689031.3696062
ISBN: 9798400711961

Deft: A Scalable Tree Index for Disaggregated Memory

Jing Wang
Tsinghua University
Beijing, China

Qing Wang
Tsinghua University
Beijing, China

Yuhao Zhang
Tsinghua University
Beijing, China

Jiwu Shu*
Tsinghua University
Beijing, China

Abstract

Memory disaggregation has become an inexorable trend in data centers and the cloud. By physically separating compute and memory resources into independent pools and connecting them with high-speed networks, memory disaggregation enables high resource utilization and elastic resource scaling. However, traditional tree-based indexes become inefficient on disaggregated memory: (1) large tree nodes can easily saturate network bandwidth due to I/O amplification, yet small tree nodes increase network round trips; (2) expensive concurrency control schemes restrict the scalability.

We present Deft, a tree-based index for disaggregated memory that delivers high throughput and scalability. Deft 1) introduces segmented internal nodes and hash-based leaf nodes for fine-grained access patterns to achieve low I/O amplification without increasing the index height; 2) proposes a scalable write-write concurrency control scheme on top of the tailored node structure with a one-sided shared-exclusive lock, improving scalability for concurrent update and insert operations; 3) proposes a lightweight and correct read-write concurrency control scheme, improving scalability for efficient lock-free search operations. Our experimental results demonstrate that Deft achieves high performance and scalability on disaggregated memory, and outperforms state-of-the-art ordered indexes by 2.4-9.5× under various workloads.

CCS Concepts: • **Information systems** → **Data structures**; • **Software and its engineering** → **Distributed memory**.

Keywords: Disaggregated Memory, Index, RDMA

*Jiwu Shu is the corresponding author (shujw@tsinghua.edu.cn).

This work is licensed under a Creative Commons Attribution International 4.0 License.
EuroSys '25, March 30–April 3, 2025, Rotterdam, Netherlands
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1196-1/25/03.
https://doi.org/10.1145/3689031.3696062

ACM Reference Format:
Jing Wang, Qing Wang, Yuhao Zhang, and Jiwu Shu. 2025. Deft: A Scalable Tree Index for Disaggregated Memory. In *Twentieth European Conference on Computer Systems (EuroSys '25), March 30–April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3689031.3696062

1 Introduction

Disaggregated memory (DM) architecture separates computing and memory resources into distinct resource pools, and connects them via a high-speed network fabric (e.g., RDMA). DM architecture has attracted extensive interest in both academia [15, 21, 34, 40, 41, 54, 55] and industry [2, 3, 7, 33, 36] due to its ability to offer high resource utilization, elastic resource scaling, and improved reliability in data centers and the cloud.

For these attractive advantages, numerous DM-based systems are emerging, including databases [7, 55] and in-memory key-value (KV) stores [21, 35, 40]. Ordered index structures (e.g., B^+Tree) are the fundamental pillar of these systems. However, with the wimpy computing power of memory nodes (MNs), the compute nodes (CNs) are required to use one-sided RDMA operations to access memory on MNs, which makes it challenging to design an efficient ordered index structure on DM:

(1) Large I/O amplification squanders the network bandwidth. Tree-based indexes on DM tend to adopt large tree nodes (fanout) for reducing the tree height and thus network round trips [41, 57]. Yet, for each index node in the search path, CNs have to fetch the whole node and search it in local memory. Employing large tree nodes leads to enlarged I/O amplification, which squanders the network bandwidth. In addition, the network fabric is shared by a considerable number of threads on CNs. As a result, the high bandwidth offered by RDMA NICs is still insufficient in DM systems.

(2) Expensive write-write concurrency control limits the scalability. For write-write conflicts, commonly used lock-based concurrency control relying on `RDMA_CAS` has poor scalability [17]. Besides, the per-tree-node lock restricts concurrent operations on the same tree node, hampering the performance under skewed workloads.

(3) Complicated read-write concurrency control incurs extra overheads. For handling read-write conflicts,

optimistic synchronization schemes (e.g., checksum and versioning) are preferred, since they enable lock-free reads and detect data inconsistency (concurrent modification). However, due to the limited guarantees on the ordering involved in one-sided RDMA operations, existing correct optimistic approaches have non-negligible overheads, such as multiple read operations or write amplification.

Existing state-of-the-art DM-based ordered indexes [24, 25, 41, 43] still suffer from some of these deficiencies, being bottlenecked by network bandwidth, delivering unsatisfactory performance under skewed write-intensive workloads, or even adopting an incorrect approach for read-write conflicts [41, 56]. We conduct an in-depth analysis in §3.

Nevertheless, it is not straightforward to overcome these challenges. A naive approach to alleviate I/O amplification is reducing the size of tree nodes. However, reducing the node size will generate more internal nodes due to the smaller fanout, which increases the tree height and the tree size. A larger tree height results in more network round trips for searching; and a larger tree size reduces the effect of index caching, which is critical for performance [41], as CNs have scarce local memory resources for caching. The same applies to using a radix tree since a radix tree has a sparse structure and a large tree height. For concurrency, sophisticated lock-free concurrency control schemes on tree-based indexes [22, 44, 50] are not suitable for DM, since they either increase write amplification (e.g., copy-on-write) or require multiple round trips to complete an operation (e.g., split operations or read multiple delta updates).

In this paper, we explore how to design a scalable ordered index on disaggregated memory. We propose Deft, a disaggregated-memory-friendly tree index. The fundamental principles underlying Deft are aimed at *reducing I/O amplification* by designing fine-grained access patterns for tree nodes without increasing index height, and *facilitating high concurrency* by employing a scalable write-write concurrency control scheme on top of the tailored node structure and a lightweight optimistic read-write concurrency control scheme.

To reduce I/O amplification, Deft designs fine-grained access patterns for tree nodes so that Deft can fetch only a portion of a node. For internal nodes that require ordering for searching, Deft introduces a segmented node structure by logically dividing a node into several unsorted sub-nodes but retaining partial order among sub-nodes. In this way, the read granularity of internal nodes is reduced from an entire node to a sub-node. Deft locates a sub-node by calculation but not an extra pointer, thus not increasing the tree size. For leaf nodes, Deft adopts a hash-based node structure, which comprises multiple hash buckets. Deft maps keys to specific buckets in leaf nodes, and thus reduces the read granularity from an entire leaf node to buckets. The write granularities of both internal nodes and leaf nodes are further reduced to entry-level owing to their unsorted structures.

To improve concurrency, Deft introduces a scalable concurrency control scheme on top of the tailored node structure. Due to the entry-level write granularity in Deft's node structures, Deft leverages `RDMA_CAS` or extended `RDMA_CAS` to upsert (update and insert) and enables concurrent upsert operations in one leaf node. To coordinate upsert operations and structural modification operations (SMOs), Deft introduces a one-sided shared-exclusive lock that has a good chance of being acquired successfully with only one round trip. With the help of this lock, Deft executes concurrent upsert in one leaf node in shared mode, providing high throughput and scalability even if under skewed workloads.

To enable efficient lock-free search, we propose on-path decoupled versioning (OPDV), a lightweight optimistic synchronization technique, for Deft. By leveraging the pointer-chasing pattern in the search path, OPDV decouples versions into the current tree node and its parent node, thus naturally validating data consistency on the search path. Compared to existing correct optimistic synchronization techniques, OPDV neither requires additional read operations nor generates write amplification.

We implement Deft and evaluate it against state-of-the-art ordered indexes on DM with a series of benchmarks. The evaluation results show that Deft achieves high scalability and better performance under various settings, outperforms Sherman [41] and SMART [25] by up to 9.5× and 2.4× under write-intensive workloads, and 4.9× and 5.9× under read-only workloads, respectively.

In summary, this paper makes the following contributions:

- We analyze and discover the bottleneck and deficiency of existing tree indexes on DM (§3).
- We introduce and implement Deft, a tree-based index for DM that delivers high performance and scalability by designing fine-grained access patterns of tree nodes, and proposing a scalable write-write concurrency control scheme and a lightweight read-write concurrency control scheme (§4).
- We evaluate Deft against state-of-the-art ordered indexes on DM to demonstrate the advantage of Deft (§5).

2 Background

2.1 Memory Disaggregation

In traditional data centers, a monolithic server tightly couples its CPUs and memory within one machine. This architecture leads to low resource utilization and also fails to scale one resource independently [12, 34].

Recently, with advances in networking technologies (e.g., RDMA and Compute Express Link (CXL)), memory disaggregation has become a prevalent architecture in data centers and the cloud [7, 23, 30, 36, 42, 53–55]. The disaggregated memory (DM) architecture physically separates CPUs and memory into two independent pools of compute nodes (CNs)

and memory nodes (MNs), and connects them with a high-speed network. The CNs are equipped with sufficient computing resources (e.g., CPUs) but limited memory, while the MNs own abundant memory resources but a few wimpy computing resources.

Owing to resource pooling, the DM architecture brings several advantages for data centers and the cloud [42, 53]. The DM architecture improves resource utilization and reduces the total cost of ownership (TCO). Moreover, DM provides elastic resource scaling of computing and memory resources, which can satisfy the dynamic resource demands of applications [7]. Finally, the disaggregated architecture avoids the problem of fate-sharing, i.e., the failure of one resource node will not affect the other nodes, achieving better availability and reliability.

2.2 RDMA Network

RDMA, a high-speed network, is increasingly deployed in modern data centers for its high bandwidth (e.g., 100 Gbps) and low latency (μs-level) [17]. Servers can communicate with each other with RDMA by using *two-sided verbs* (e.g., `RDMA_SEND`, `RDMA_RECV`) or *one-sided verbs* (e.g., `RDMA_READ`, `RDMA_WRITE`, and `RDMA_ATOMIC`). As one-sided verbs allow a server to directly access the memory on another server without involving the CPUs of the remote server, they are preferred for the DM architecture that has scarce computing resources in memory nodes [24, 25, 35, 41, 51, 57, 58].

3 Motivation

Designing an efficient ordered index on DM is challenging due to the characteristics of DM systems and RDMA networks. In this paper, we focus on B$^+$Tree on DM for its universality. Nevertheless, some of our ideas can be applied to other kinds of index structures, such as learned index [24], radix tree [25], and LSM-tree [43].

Sherman [41] is a state-of-the-art B$^+$Tree on DM that combines RDMA hardware features and software techniques to boost write performance. However, we found that tree-based indexes including Sherman are still overshadowed by three issues on disaggregated memory.

3.1 Dilemma of I/O Amplification and Round Trips

As memory nodes in the DM architecture have wimpy computing resources, most index operations have to be performed using one-sided RDMA verbs. The characteristics of accessing remote memory with one-sided RDMA verbs differ significantly from accessing local memory. For local memory, the access latency is low and the access granularity is typically a cache line (e.g., 64 bytes), and sequential accesses are streaming due to the cache line prefetching. By contrast, for indexes on remote memory, the access granularity is the node size since the entire node must be fetched to local memory before operating on it. Besides, the bandwidth

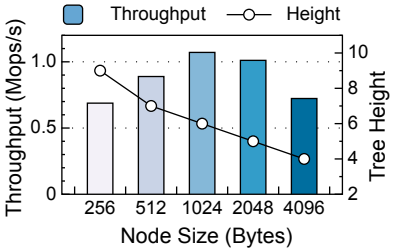

Figure 1. The read performance of Sherman with different node sizes.

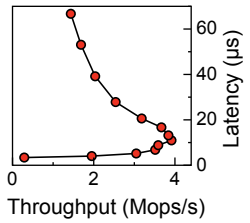

Figure 2. Lock performance.

of local memory (e.g., 20 GB/s per DIMM) is much larger than RDMA networks (e.g., 12.5 GB/s for a 100 Gbps NIC).

Tree-based indexes are designed with a high node fanout (e.g., 1KB node size in Sherman [41]) and thus a low tree height to reduce excessive network round trips. However, this leads to large I/O amplification and easily saturates the network bandwidth. Theoretically, the tree height is $\log_F N$, and the I/O size to traverse the tree is $O(F \cdot \log_F N)$, where N is the total number of keys and F is the fanout of the tree. While reducing the node size (F) is a straightforward way to alleviate I/O amplification, it comes at the cost of increased tree height and thus more round trips for index operations. The same applies to using other indexes that have smaller node sizes such as radix trees.

To show the impact of I/O amplification, we conduct an experiment on Sherman with an MN and a CN with 16 client threads. Each server is equipped with a 100 Gbps Mellanox ConnectX-5 RNIC. Figure 1 shows the read throughput and tree height of Sherman with different node sizes when disabling the CN-side index cache. Even with only one CN, the RNIC bandwidth is saturated when the node size is larger than 1 KB, and thus the throughput drops and the latency increases. Reducing node size results in a higher tree height and therefore more network round trips. Though CN-side index cache can help reduce network round trips, the internal part of the tree gets larger with a smaller fanout (the size of the internal part is $O(\frac{N}{F-1})$), which lowers the hit ratio of the size-limited cache.

3.2 Poorly-Scaled Write-Write Concurrency Control

One advantage of DM systems is the ability to elastically scale the CNs and accommodate a large number of concurrent requests. However, this poses challenges for DM systems due to the high cost of cross-network concurrency control.

We conduct another experiment to show the performance issue of lock-based concurrency control, which is commonly adopted in tree-based indexes for write-write conflicts [24, 41, 57]. In this experiment, up to 96 threads across 3 CNs execute `RDMA_CAS` to acquire and release one of 10240 remote locks on an MN. The access pattern follows Zipfian distribution with a skewness parameter of 0.99. Following previous work [41], the remote locks reside in the on-chip memory of RNIC for

better performance, which avoids PCIe transactions from RNIC to host memory at the MN side.

As shown in Figure 2, though residing in the on-chip memory, the remote lock has poor scalability and gets a peak throughput lower than 4 Mops/s at 40 threads. The performance further drops with the number of threads increasing. This is because acquiring a remote exclusive lock requires multiple retries (RDMA_CAS) due to the high contention and each retry involves a cross-network round trip. Besides, excessive retries queue up in the RNIC and consume RNIC IOPS, which results in higher latencies and lower throughput. Recent work proposes designs to reduce the number of remote retries over RDMA, such as hierarchical lock [41] or MCS queue lock [5]. They help but not thoroughly due to the inherent exclusion of locks. Since a remote lock is held to protect a much larger critical section (cross-network operations), the node-level lock restricts concurrent operations and provides low performance under skewed workloads.

3.3 Inefficient Read-Write Concurrency Control

Besides write-write concurrency control, enabling concurrent and correct read-write operations is also critical for scalability. Pessimistic synchronization techniques (e.g., lock-based) are easy to implement but do not scale well. For high-degree concurrency, indexing systems on DM prefer employing optimistic approaches to enable lock-free search without holding a lock.

However, implementing a correct and efficient optimistic synchronization scheme is not straightforward. As pointed out by a recent study [56], some previous works [28, 41] realize synchronization schemes with a common but incorrect assumption that RDMA_READ is guaranteed to be performed in increasing address order, which results in hard-to-detect data inconsistencies. For example, Sherman [41] stores a pair of versions at the beginning and end of data (i.e., a tree node) and assumes they can be read by RDMA_READ in increasing address order. In fact, the InfiniBand specification [4] does not provide any guarantee on the ordering of RDMA_READ and the PCIe specification [32] allows read operations to fetch multiple cache lines concurrently.

There exist three correct optimistic synchronization techniques based on limited ordering guarantees [56].

Checksum. Each data object is associated with a checksum (e.g., CRC). When updating an object, writers re-calculate the checksum and update it together with the object. Readers verify an object by re-calculating the checksum and comparing it with the checksum stored in the object.

Versioning. Versioning is a widely used technique in traditional local memory systems. Writers increment the version number of an object before and after updating data. Readers need to issue three individual RDMA_READ operations for an object: read the version, then the object, and then the version again for validation. Readers ensure there are no concurrent updates if the two version numbers are identical.

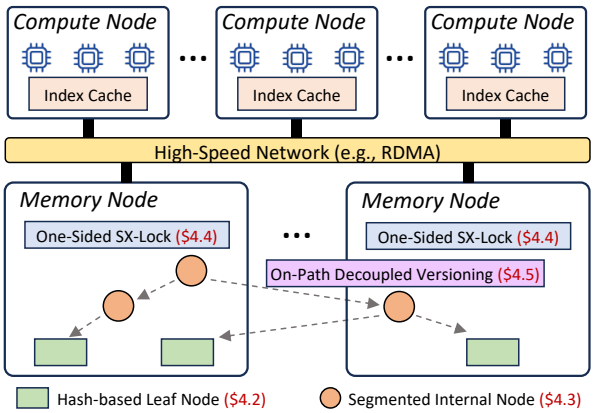

Figure 3. The overall architecture of DEFT.

FaRM cache line versions. FaRM [13] proposes a technique that embeds a version number at the beginning of each cache line occupied by an object. A writer needs to first update all cache line versions to a special value, then update the data, and finally update the versions again to a committed value. Readers fetch the object with one RDMA_READ and detect consistency by validating that all version numbers match and are not the special value.

Nevertheless, none of the three techniques are ideal for DM systems. *Checksum* is computationally expensive[1] and restricts the RDMA I/O granularity to a full data object for re-calculation. *Versioning* requires *three* individual RDMA_READs for fetching an object, which cannot be combined or overlapped since read operations can be re-ordered (e.g., in PCIe layer [32, 56]). *FaRM* approach is not friendly to remote writers as it requires multiple writes to update versions and data. Note that when updating versions, as version numbers scatter across all cache lines the object occupies, writers have to write the entire object with RDMA_WRITE, which results in write amplification and consumes network bandwidth.

4 DEFT Design

4.1 Overview

Motivated by the analysis above, we propose DEFT, a scalable distributed B$^+$Tree on disaggregated memory. The design of DEFT is guided by two primary goals: *(1) mitigating I/O amplification to optimize network bandwidth utilization*, and *(2) enhancing scalability to accommodate a large number of concurrent operations.*

Figure 3 shows the overall architecture of DEFT. The tree nodes are distributed across the disaggregated memory pool. Client threads at CNs use one-sided RDMA operations to access the tree index. Like previous work, each CN has a size-limited index cache for internal nodes to facilitate tree searching and reduce network round trips. To achieve the

[1] In our platform, calculating a 32-bit CRC for 1KB data via Boost library costs about 3μs.

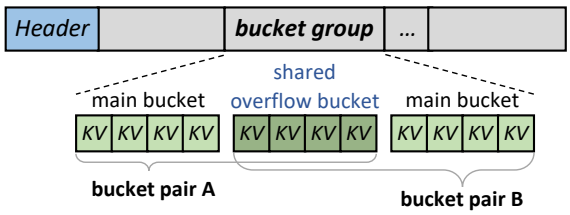

Figure 4. The structure of hash-based leaf node.

first goal, DEFT redesigns access patterns of leaf nodes and internal nodes to reduce the access granularity, without altering the tree size or increasing the tree height (§4.2 and §4.3). For the second goal, DEFT introduces a scalable concurrency coordination scheme with a one-sided shared-exclusive lock to support highly concurrent and skewed upsert (update and insert) operations (§4.4). DEFT further proposes an on-path decoupled versioning to enable lock-free search without introducing extra overheads (§4.5).

4.2 Hash-based Leaf Node

Traditional RDMA-based B$^+$Tree has to fetch the entire leaf node to search for the target key [41, 57], which has high I/O amplification and squanders the network bandwidth. To address this issue, we propose the hash-based leaf node to reduce the I/O granularity.

Figure 4 shows the structure of the hash-based leaf node. Similar to a typical leaf node, the leaf node in DEFT consists of a header and a KV entry array, and is in charge of a small range of keys. The difference is that the KV entry array is divided into multiple buckets, and KV entries are stored in specific buckets according to the key's hash value. To reduce hash conflicts, each bucket has multiple slots to handle conflicting KV entries (i.e., associativity). Additionally, each bucket has an overflow bucket to further accommodate conflicting KV entries. Inspired by the overflow colocation scheme in RACE hashing [58], two main buckets share one overflow bucket in a bucket group, and the overflow bucket is located between the two main buckets. A main bucket and its overflow bucket constitute a *bucket pair*. Thus, a bucket pair can be fetched by one `RDMA_READ` operation. Due to the associativity and overflow buckets to tolerate more collisions, the hash-based leaf node can achieve comparable memory utilization as B$^+$Tree (i.e., ~70%). Different from Blink-hash [9], a recent work that also adopts hash leaf nodes in a tree-based index to reduce contention for monotonic insertions, the main purpose of DEFT's hash-based leaf node is to reduce the I/O granularity.

4.2.1 Search and Upsert in Leaf Nodes.

When searching for a KV entry in the leaf node, instead of fetching the entire leaf node, DEFT only fetches the target bucket pair (i.e., the main bucket and the overflow bucket) by one `RDMA_READ` to reduce the read amplification. DEFT uses node-level synchronization (§4.5) to detect split and data inconsistency.

For update operations, DEFT only needs to update the value. As `RDMA_WRITE` does not have any atomicity guarantees [14, 32], to prevent concurrent readers from observing torn data, DEFT updates the value[2] via 8-byte `RDMA_CAS` directly to ensure data atomicity.

For insert operations, DEFT finds an empty slot and only writes back the KV entry atomically to reduce the write amplification via extended `RDMA_CAS` provided by widely-deployed Mellanox RDMA devices [39]. Extended `RDMA_CAS` supports atomic compare-and-swap up to 32 bytes. For large-sized keys that cannot be updated by extended `RDMA_CAS` or RNICs that do not support extended `RDMA_CAS`, DEFT first writes a specific lock value to the slot of value via 8-byte `RDMA_CAS`. If the CAS succeeds, DEFT writes the key, and then atomically updates the value to the actual value. Thus, readers can ignore incomplete entries whose value is a void or lock value.

For range queries, DEFT fetches the entire leaf node and searches required KV entries.

4.2.2 Structural Modification Operations.

In a traditional B$^+$Tree, a leaf node split is triggered when the leaf node is full. By contrast, the leaf node in DEFT triggers a split operation when the hash table needs resizing (i.e., one main bucket and its overflow bucket both are full). During split operations, the thread fetches the entire leaf node, sorts the KV entries, and moves half of the KV entries to a new leaf node. As concurrent readers may fetch an inconsistent leaf node during the split, DEFT adopts a lightweight synchronization scheme for node-level data consistency (§4.5).

4.3 Segmented Internal Node

Different from leaf nodes, the internal node has to support ordered searching to find the correct child node. DEFT introduces segmented internal nodes to reduce the I/O amplification, without increasing the number of nodes. DEFT adopts a similar idea to learned index [19]. The key range of an internal node is evenly divided into several sub-ranges, and DEFT reorganizes KV entries into corresponding sub-nodes. When searching for a target key, threads can fetch the target sub-node whose range contains the target key. Thus, the read granularity is reduced from an entire node to a sub-node.

Figure 5 shows an example of an internal node with key range [100, 500) that is divided into 4 sub-nodes. For a 1KB node, DEFT first tries to divide the key range into 4 sub-nodes by default, and merges adjacent sub-nodes to a larger one if any sub-node becomes full. Though uneven key distribution can affect the effectiveness of our approach as learned indexes, keys in internal nodes at upper levels are subsets of keys of lower levels and thus are distributed more evenly.

Note that this design is not equivalent to directly reducing node size, since it neither reduces the node's fanout

[2] Similar to prior RDMA-based indexes [25, 41, 58], for variable-length value, the leaf node stores a pointer to the full KV items stored outside the index.

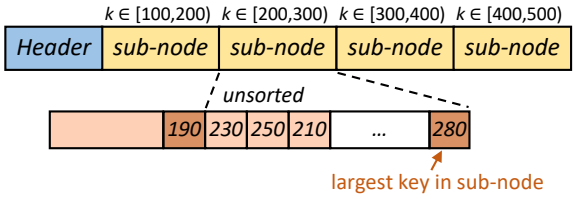

Figure 5. The structure of segmented internal node.

nor adds extra pointers, and thus does not increase the tree height. By contrast, using a smaller internal node will reduce the fanout and generate more key-pointer entries in parent nodes, increasing the tree height and thus reducing the overall performance as analyzed in §3.1.

Different from typical internal nodes, entries within each sub-node are unsorted, which avoids shifting existing entries upon insert operations. In each sub-node, the largest key is maintained at the rightmost slot for searching purposes.

4.3.1 Search in Sub-Nodes. Though the sub-nodes are unsorted, searching for the target key locally has far less overhead than fetching the sub-node by RDMA_READ. To locate the child node that contains the target key K, Deft searches for the largest key not greater than K without sorting. This key might be the largest key in the left sub-node. For example, when we are searching for K 205 in Figure 5, we locate the sub-node starting with key 200, but the correct child node is the node with key 190 (i.e., the largest one in the left sub-node). Thus, Deft fetches the current sub-node with the largest key in the left sub-node (i.e., the rightmost slot in the left sub-node) by one RDMA_READ.

If the child node is still an internal node, Deft needs to infer the target sub-node within the child node to reduce the granularity of RDMA_READ. As Deft does not include a separate pointer for each sub-node, Deft locates a sub-node directly via the key range of a sub-node. With the key range $[K_{MIN}, K_{MAX})$ of the child node (e.g., [100, 500) in Figure 5), Deft can easily calculate the key range of each sub-node. Deft gets the information of K_{MIN} and K_{MAX} in the current sub-node: the K_{MIN} is just the key of the child node, and the K_{MAX} is the smallest key larger than K. The number of sub-nodes (i.e., granularity) is stored in the unused bits of the pointer to the child node.

One corner case is that the K_{MIN} is the largest key in the current sub-node (note that the current node is also read partially), so Deft cannot get the K_{MAX} of the child node. In this case, Deft falls back to read the entire child node.

4.3.2 Insert in Sub-Nodes. In a traditional internal node, an insert may cause a shift operation. Since RDMA_WRITE is non-atomic, previous work uses node-level synchronization techniques (e.g., checksum or versioning) to protect data consistency for lock-free reads (§3.3). By contrast, Deft adopts unsorted sub-nodes to avoid expensive node-level synchronization and further reduce the write amplification of insert operations. As shown in Figure 5, the internal node is partially ordered but sub-nodes are unsorted.

When inserting a new key not larger than the largest key in the current sub-node, Deft finds an empty slot and atomically inserts the key-pointer pair via extended RDMA_CAS as in leaf nodes (§4.2.1). To keep the rightmost slot containing the largest key, if the new key is larger than the current largest key, Deft swaps the current largest key to an empty slot and inserts the new key into the rightmost slot.

4.3.3 Sub-Node Merge Operations. With new keys continuously inserted, if a sub-node is full, Deft merges every two adjacent sub-nodes to a larger sub-node in the current internal node. Owing to the feature of unsorted sub-nodes, Deft does not need to shift any KV entries; and owing to the feature of partial-ordered internal nodes, the largest keys in the original right sub-nodes are still the largest keys of the new sub-nodes. Therefore, Deft only needs to update the granularity in the header and the pointer in the parent node. As the merge operation in the internal node is infrequent, this extra overhead is negligible.

4.3.4 Structural Modification Operations. When the internal node is full, Deft splits the internal node as a typical B^+Tree. The difference is that new internal nodes in Deft are organized as multiple sub-nodes for small I/O amplification.

4.4 Scalable Concurrency Coordination

The exclusive-lock-based concurrency control for write-write conflicts has been commonly used in tree-based indexes. Nevertheless, as analyzed in §3.2, the exclusive lock manipulated with RDMA_CAS has poor scalability and severely limits the performance of DM systems. In this section, we first introduce the concurrent upsert scheme in Deft (§4.4.1) with the support of a one-sided shared-exclusive (SX) lock (§4.4.2). Then, with the SX lock, we propose RDMA operation combinations to further reduce network latencies (§4.4.3).

4.4.1 Upsert without Exclusive Lock. As described in §4.2.1, Deft updates or inserts KV entries via (extended) RDMA_CAS for data atomicity. Besides, leveraging RDMA_CAS can handle write conflicts for upsert operations. Moreover, the hash-based leaf node provides a low collision rate for multiple insert operations. In hash-based leaf nodes, new insertions are dispersed to multiple buckets, neither causing shift operations as in a traditional sorted leaf node nor contending for the tail position as in a traditional unsorted leaf node. Thus, the hash-based leaf node allows concurrent upsert operations in one leaf node to be executed simultaneously with a high success rate.

SMOs (e.g., split) need to be executed exclusively. Thus, an efficient shared-exclusive lock is required to support the concurrency coordination. A writer first acquires a shared lock and tries to insert or update the KV entry optimistically

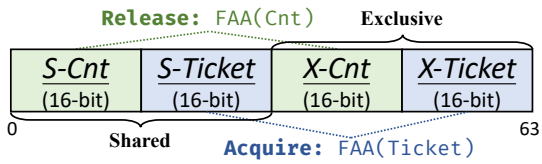

Figure 6. The format of the shared-exclusive lock.

in a lock-free manner (without exclusive lock). If the tree node is full, the writer upgrades its lock from shared mode to exclusive mode and executes the SMO.

One issue with lock-free-manner insertion is that different threads may concurrently insert an identical key at different slots if there exists a concurrent delete operation [10]. To address this issue, previous works [10, 58] re-scan the bucket after the insertion and delete the duplicate keys. However, the additional re-read operation incurs high network latency in RDMA-based systems. To avoid this issue, DEFT makes delete operations to be executed in the exclusive mode. This will not hinder the concurrent performance much since delete operations are infrequently in practice according to recent real-world workloads analysis [8, 48].

4.4.2 One-Sided Shared-Exclusive Lock. To support the above concurrency coordination, we propose an efficient shared-exclusive (SX) lock that can be acquired with only one `RDMA_ATOMIC` in the ideal case. Like previous work [41], we separate locks from tree nodes, and each tree node is associated with a lock by hashing its address.

We design the SX lock following Lamport's bakery algorithm [20]. Specifically, as shown in Figure 6, each lock is a 64-bit integer and consists of 2 pairs of counters (`Cnt`) and ticket numbers (`Ticket`) for a shared mode and an exclusive mode, respectively. Threads manipulate the SX lock via extended masked `RDMA_FAA` [39] that supports the masked semantic and can add values on multiple fields independently. Different from previous work [49] that is also based on Lamport's bakery algorithm, DEFT uses the masked `RDMA_FAA` and thus does not have the risk that 16-bit fields (i.e., `Cnt` or `Ticket`) overflow to their preceding fields. For RNICs that do not support masked atomic operations, the counters should be reset periodically as in the previous work [49].

Specifically, to acquire a shared lock, a thread uses masked `RDMA_FAA` increments corresponding `S-Ticket` and fetches the original value of `X-Ticket` and `X-Cnt`. If `X-Ticket` equals `X-Cnt`, which means no exclusive lock is held, the thread acquires the shared lock successfully. Otherwise, the thread uses `RDMA_READ` to poll `X-Cnt` until it matches the previously fetched value of `X-Ticket`.

To acquire an exclusive lock, a thread increments `X-Ticket` and fetches original values of `X-Ticket`, `X-Cnt`, `S-Ticket`, and `S-Cnt`. If `S-Ticket` equals `S-Cnt`, and `X-Ticket` equals `X-Cnt`, which means no shared locks and exclusive locks are held, the thread acquires the exclusive lock successfully. Otherwise, the thread polls `S-Cnt` and `X-Cnt` until currently held locks are released (i.e., match `Ticket`).

To release a shared lock or an exclusive lock, a thread increments `S-Cnt` or `X-Cnt` by `RDMA_FAA`, respectively.

Thus, in ideal cases, both the shared lock and the exclusive lock can be acquired with one `RDMA_FAA`. Owing to the bakery algorithm, both the shared lock requester and the exclusive lock requester will only wait for a finite and fixed number of locks to be released when they get their ticket numbers. Hence, the shared-exclusive lock will not cause starvation for lock acquisition.

When a thread needs to upgrade a shared lock to an exclusive lock for SMOs, it releases its shared lock and acquires the exclusive lock simultaneously with one `RDMA_FAA` operation. Specifically, with the masked `RDMA_FAA` operation, threads can increment `S-Cnt` and `X-Ticket` at the same time. Thus, one round trip for releasing the shared lock is saved.

4.4.3 RDMA Operation Combination. Owing to the ordering property of reliable connected (RC) mode provided by RDMA NICs [4], DEFT combines continuous RDMA operations that obey the ordering rules to reduce network round trips. For example, in write operations, DEFT combines the entry write back (`RDMA_CAS`) and the following lock release (`RDMA_FAA`) in one round trip.

Further, in write operations, DEFT combines the lock acquisition (`RDMA_FAA`) and read page (`RDMA_READ`) optimistically. The drawback of this optimization is that if the lock acquisition fails, the combined `RDMA_READ` is invalid and wastes network resources. However, due to the hash-based leaf node and the concurrent upsert scheme, most locks are acquired in shared mode and have a high success rate, which improves the effectiveness of the combination. Moreover, in the worst case where multiple threads request the same exclusive lock, a thread can determine how many exclusive locks will be acquired before this thread with the ticket number and current counter. Thus, to save network bandwidth, threads do not prefetch the read if they find the length of the waiting queue (i.e., `Ticket - Cnt`) is large during the retry process.

Previous work [17, 41, 51] tends to use *Doorbell batching* to combine and issue multiple RDMA operations to a QP, and the RNIC fetches WQEs from host memory via DMA. Though this technology reduces CPU-initiated MMIOs, it is not efficient for small RDMA operations. This is because modern RNICs can write WQE along with the Doorbell via one MMIO (e.g., *Blue Flame* of Mellanox NICs which is enabled for small-sized WQEs by default [31, 38]), and small payload can be inlined in the WQE (e.g., `RDMA_ATOMIC` and `RDMA_WRITE` with small values). Moreover, the NIC-initiated DMA has a higher latency than the CPU-initiated MMIO. Thus, for two continuous small RDMA operations (e.g., the cases described above), DEFT doesn't use Doorbell batching. Instead, DEFT posts the first RDMA operation asynchronously without signaling completion, then posts the

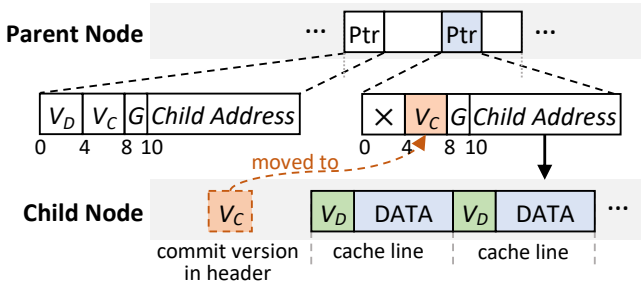

Figure 7. Node layout with OPDV.

second RDMA operation immediately and waits for the completion. The completion of the second RDMA operation also indicates the completion of the first RDMA operation, since the completion ordering in the same QP obeys the ordering they were submitted [4].

4.5 On-Path Decoupled Versioning

Deft proposes on-path decoupled versioning (OPDV), a lightweight and correct optimistic synchronization scheme, to enable lock-free search operations for further improving the concurrency performance. Deft adopts OPDV only for node-level consistency (e.g., split), since entry-level consistency (i.e., insert and update) is guaranteed by `RDMA_CAS`.

As analyzed in §3.3, the versioning approach requires three `RDMA_READ`s to fetch an object, and the FaRM cache line approach has a write amplification issue. By combining the versioning approach and the FaRM cache line approach, we come up with a new synchronization scheme that only requires two `RDMA_READ`s to fetch an object and does not have the write amplification issue; by leveraging the pointer-chasing scheme of the search path, we decouple versions into the current node and its parent node, further reducing one read operation for fetching each object.

Specifically, as shown in Figure 7, Deft stores a data-side version (V_D) at the beginning of each cache line as in the FaRM cache line approach, and a commit version (V_C) in the header. When a node triggers an SMO, the writer first updates the data with new V_D by one `RDMA_WRITE`, and then updates the commit version V_C to the new V_D. For readers, they first read the V_C and then read the data with V_Ds. Note that the two read operations can not be combined or overlapped due to the out-of-ordered PCIe reads [32]. The data is consistent if the V_C and V_Ds match. Since `RDMA_WRITE` obeys the PCIe rule that writes data in increasing address order [32], the V_D in each cache line is written before the data. Thus, if readers fetch inconsistent data, they will observe V_Ds different from V_C.

To further eliminate the `RDMA_READ` for V_C, OPDV moves the V_C from the header to the child pointer that points to the current node in the parent node. In this way, we can directly get the V_C in the search path and save an `RDMA_READ`. Besides, to reduce the storage overheads of V_D, Deft stores the value in front of the key in each key-value pair, and embeds the V_D in the pointer. Putting it all together, as shown in Figure 7, a pointer in internal nodes consists of a 4-bit V_D for the current cache line (only the first 4-bit in a cache line), a 4-bit V_C for the child node, a 2-bit read granularity (G) for sub-nodes (§4.3), and the address to the child node.

4.6 Discussion

Failure handling. A crash during an SMO may result in deadlock, memory leakage, and data inconsistency. Similar to a B$^+$Tree, Deft only changes the layout of KV entries in tree nodes and retains similar write operations to the memory pool. Thus, recently proposed solutions to failure handling should be applicable to Deft.

For handling deadlock, timeout-based approaches (e.g., lease) are effective solutions used by previous work [46, 49]. Each acquired lock has an expiration time (i.e., lease time), which is large enough for an insert or an SMO operation. If a client with a lock crashes, which means it is unable to release its lock, other clients can detect the deadlock by continuously getting the same count values (`S-Cnt` or `X-Cnt`) for a long period (e.g., twice the lease time). Then, the memory node or other clients can reset the lock value to release the deadlock.

For other corruption, FUSEE [35] proposes an embedded operation log scheme to resolve metadata corruption, including memory allocation information for handling memory leakage. When a client crashes, the operation log can be used to resume the client's partially executed operations, and inconsistent data can be resolved with the help of replication. Besides, rTX [47] proposes transactional indexes on disaggregated memory that each index operation can be failure atomic: Non-atomic operations can be repaired with the help of logging. We leave failure handling to future work.

Generality of Deft's techniques. Some principles of Deft are general and can be applied to other DM systems and are not limited to index structures. First, for the I/O amplification issue, systems can break the stereotype of sorted data and design a fine-grained access pattern. This is because the overhead of computation (e.g., sorting) is much smaller than remote accesses. Second, fine-grained concurrency control is necessary for highly concurrent operations. Third, the OPDV can be applied to many other DM systems, since the pointer-chasing pattern is quite common in many systems.

Generality to future disaggregated memory. In this paper, we focus on RDMA-based disaggregated memory. Future DM architecture can be deployed with more advanced connections, such as CXL [1], which may provide higher bandwidth and lower latency than current RDMA. However, according to recent studies [6, 23, 26, 37, 52], in-server CXL-memory has high access latency, reaching several hundred nanoseconds (e.g., 250-485 ns [37]), let alone switch-connected remote CXL-memory. Searching a KV entry in

a large tree node (e.g., binary search) requires issuing multiple memory access instructions, leading to high latencies. Thus, DEFT's fine-grained access patterns are still valuable for limiting the searching scope, thus reducing the number of memory access instructions to remote memory.

Besides, the higher-than-local memory latency also leads to high overheads of cache coherence, which results in low throughput of atomic operations [52]. Thus, DEFT's concurrency control scheme and OPDV can reduce synchronization overheads and provide better scalability.

5 Evaluation

In this section, we evaluate DEFT against the state-of-the-art ordered indexes for disaggregated memory to show:

- The overall performance of DEFT (§5.2).
- The effect of DEFT's techniques to performance (§5.3).
- The impact of workload characteristics to DEFT (§5.4).
- The end-to-end performance of DEFT (§5.5).

5.1 Experimental Setup

5.1.1 Hardware Platform. We conduct all experiments on a cluster with 10 servers. Each server is equipped with a 2.6 GHz Intel Xeon 6240M CPU (18 physical cores), 96 GB DRAM, and a 100 Gbps Mellanox ConnectX-5 NIC, and runs Ubuntu 18.04 LTS with Linux kernel 4.15. The version of drivers for the RDMA NICs is OFED-4.9-5.1.0.0. All RDMA NICs are connected with a 100 Gbps IB switch. Due to the limited size of the cluster, each MN is colocated with one CN to save machine resources like previous work [25, 41].

Workload	Feature	Read-Write-Scan %
A	update-heavy	50-50-0
B	read-mostly	95-5-0
C	read-only	100-0-0
D	read-latest	95-5-0
E	scan-intensive	0-5-95

Table 1. YCSB workloads description.

5.1.2 Workloads. We evaluate DEFT and other compared systems with YCSB workloads [11] in §5.2-§5.4, and Twitter workloads [48] in §5.5. Table 1 shows the characteristics of the YCSB workloads. The workload contains 400 million keys (8-byte key and 8-byte value) with the default skewed distribution (Zipfian with parameter 0.99, and latest for YCSB-D). We evaluate other settings in §5.4. We perform each workload after randomly preloading 400 million keys (Load).

5.1.3 Compared Systems. We compare DEFT with state-of-the-art indexes on DM, Sherman [41], dLSM [43], and SMART [25]. Sherman is a write-optimized B$^+$Tree on DM, dLSM is an LSM-based index on DM, and SMART is an adaptive radix tree on DM. Since Sherman adopts an incorrect synchronization technique [56], besides the original Sherman, we implemented a correct version of Sherman with the *checksum* synchronization technique (denoted as Sherman-C), as the checksum approach does not make intrusive modification to Sherman's structure (§3.3).

For all compared systems, each CN owns 1 GB DRAM for index cache. Like Sherman, DEFT puts locks in the on-chip memory of RNICs on MNs. The size of the leaf node and the internal node in both DEFT and Sherman is 1 KB (the default value in Sherman). In the current implementation of DEFT, a hash bucket in leaf nodes contains 4 slots (64 bytes), and a minimum sub-node in internal nodes contains 15 slots (240 bytes). For DEFT, Sherman, and SMART, each MN has 1 CPU core for network connection and memory allocation. Since dLSM needs computing resources on MNs for compaction, we assign 8 threads at each MN for dLSM. We enable block-based SSTables in dLSM in our experiments[3], and other configurations as default in their paper.

5.2 Overall Performance

We first evaluate DEFT and other compared indexes on YCSB workloads with 2 memory nodes and 10 compute nodes. Client threads are evenly distributed on the 10 compute nodes with 30 CPU threads per node. Except for dLSM which does not support coroutines, other indexes use coroutines to simulate more client threads. A coroutine switches to another coroutine after it issues an RDMA operation to hide the network latency. The total number of client threads scales to 900 by default, and up to 1800 if the throughput is not saturated. Figure 8 shows the throughput and latency of these indexes with the default skewed distribution.

Insert workloads. For the Load workload, dLSM achieves much higher throughput than other systems. This is because dLSM batches small writes in local MemTables and flushes them to remote memory nodes via large RDMA writes, which significantly reduces the number of RDMA writes. However, the side-effect of batching is that other CNs cannot read the newly inserted data, so dLSM adopts a range-sharding scheme on multiple CNs. DEFT has higher insert performance than Sherman and SMART due to the design of fine-grained access in both leaf nodes and internal nodes. SMART delivers the worst performance in our experiment and is not as superior as in their paper [25]. This is because our experiments employ a larger key space (400 million), which causes SMART to generate many more internal nodes and reduces the probability of path compression. The huge index size further makes the CN-side index cache less effective, and thus increases the number of RDMA reads. The checksum approach increases the latency of Sherman-C when the thread number is small. But when the thread count is large, the

[3]dLSM [43] proposes a byte-addressable SSTable layout that stores the address for every key-value pair. However, it generates a large index block for each SSTable and fails to run with small key-value pairs.

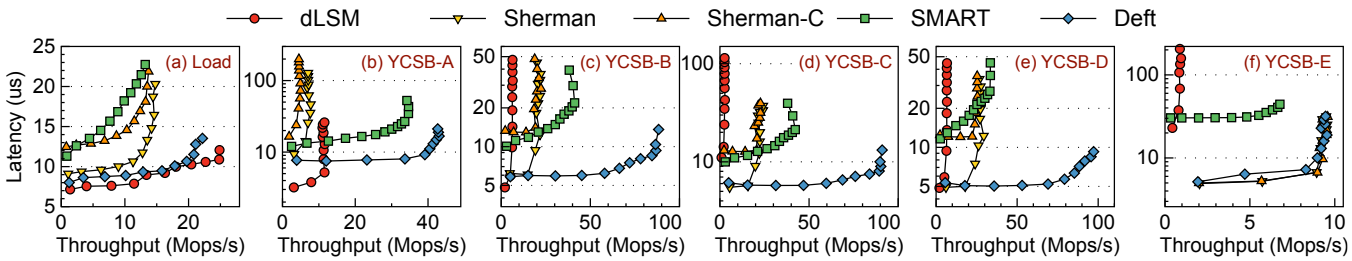

Figure 8. The throughput and latency under YCSB workloads (skewed distribution).

performance of both Sherman and Sherman-C is restricted by network bandwidth.

In this experiment, about 90% of internal nodes in Deft are automatically segmented into 4 sub-nodes. The load factor (i.e., memory utilization) of leaf nodes in Deft is 69%, which reaches the same level as Sherman.

Update workloads. For the YCSB-A workload (Figure 8(b)), Deft achieves up to 42.7 Mops/s with 900 client threads, and outperforms dLSM, Sherman, Sherman-C, and SMART in throughput by up to 3.7×, 6.1×, 9.5×, and 1.3×, respectively. This can be attributed to a combination of all techniques in Deft, especially the hash-based leaf node and the scalable concurrency coordination, which reduces the I/O amplification and relieves the contention.

Though SMART also scales well owing to its small I/O amplification, it has lower throughput than Deft. This is because SMART has a large tree height which cannot be fully cached by the index cache, and requires more RDMA round trips for searching.

Sherman does not scale well, not only due to its large I/O amplification but also because of the high contention on remote locks, even though Sherman adopts a hierarchical lock (i.e., maintains a local lock table in each compute node to reduce unnecessary remote lock retries) [41]. Note that Deft also can be enhanced with the hierarchical lock to reduce the RDMA traffic for lock, but we do not enable this technique in this paper to show the benefit of our scalable concurrency coordination. The one-sided SX lock in Deft can enable concurrent update operations in shared mode and reduce the retries on exclusive locks.

Due to the checksum approach, Sherman-C has to write back the whole leaf node with the new checksum other than only the KV entry as in Sherman. Thus, adding the checksum synchronization technique reduces the performance of Sherman by about 35%.

The performance of dLSM under the YCSB-A workload is slowed down by read operations.

Deft also has lower latency than Sherman and SMART with small numbers of threads, where the contention is not intense and the network bandwidth is not saturated yet. This is because Deft optimistically combines the lock and read operations to reduce network round-trip latencies.

Read workloads. For read-intensive workloads (YCSB-B, C, D), Deft has much higher throughput and better scalability than other compared systems owing to small I/O granularity and negligible read-write concurrency control overhead. Specifically, Deft achieves up to 91.0 Mops/s, and outperforms dLSM, Sherman, and SMART by up to 34.2×, 3.7×, and 2.2× under the YCSB-C workload.

Sherman, Sherman-C, and dLSM are all bottlenecked by the network bandwidth when the number of threads is large. Sherman does not scale after about 270 threads, because it saturates the network bandwidth of MNs due to the large I/O amplification. Even though dLSM can keep multiple MemTables in clients, dLSM has poor read performance due to the LSM-tree structure and the range-sharding scheme. First, dLSM may need to read multiple SSTable blocks via `RDMA_READ` to find the target key-value pair, even with Bloom filters. Second, dLSM adopts a range-sharding scheme that partitions keys to different CNs. Though the sharding scheme can achieve a high insert performance by batching in clients, it has the drawback of load imbalance under skewed workloads. Third, the read granularity (i.e., block of SSTable) has large read amplification and saturates the network bandwidth easily. SMART can scale well owing to its small I/O amplification. However, due to the large index size, SMART has a lower throughput than Deft as it requires more RDMA round trips for searching and finally saturates the IOPS of RNICs with a large number of client threads.

Scan workloads. We evaluate the scan-intensive workload (YCSB-E) with a scan length of 100. As shown in Figure 8(f), Deft has similar scan performance to Sherman, since Deft needs to fetch entire target leaf nodes in scan operations just as Sherman. Besides, both leaf nodes of Deft and Sherman are unsorted. Thus, they need to scan the entire leaf node to find the target key-value pairs. Deft and Sherman fetch multiple leaf nodes in a batch manner to hide the network latency. With large RDMA reads, the network bandwidth becomes the bottleneck from about 90 client threads.

Both dLSM and SMART are not good at scan operations: dLSM builds multiple iterators on MemTables and SSTables at every level, generating many RDMA reads; SMART needs to read multiple tree nodes due to its sparse structure.

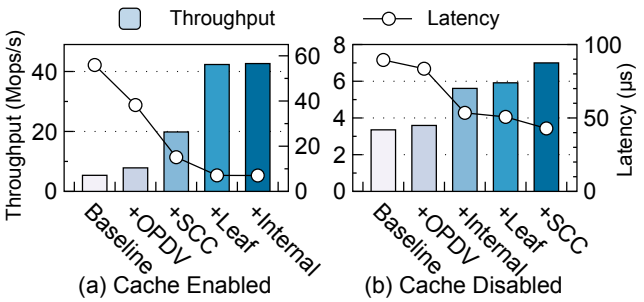

Figure 9. Contributions of techniques to performance.

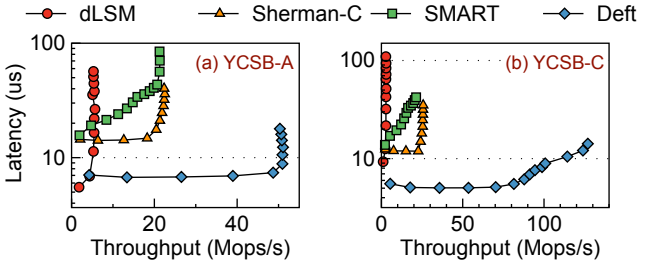

Figure 10. The throughput and latency under YCSB-A and YCSB-C workloads with uniform distribution.

5.3 Ablation Study of Techniques

In this section, we analyze the performance benefit of each technique in DEFT by applying them one by one. We conduct this experiment with 2 MNs and 300 client threads on 10 CNs, and use the YCSB-A (read 50% and write 50%) workload. Since the index cache will filter remote accesses to internal nodes, to show the benefit of the segmented internal node when the cache is limited, we also run the experiment with the index cache disabled. The two situations have different bottlenecks, so we apply techniques in different orders. The results are shown in Figure 9. The ***Baseline*** without any technique of DEFT represents Sherman-C. The ***OPDV*** represents the optimistic read-write concurrency control technique OPDV (§4.5). The ***SCC*** represents the scalable write-write concurrency coordination, which includes the concurrent upsert scheme with the one-sided shared-exclusive lock and optimistic RDMA operation combination (§4.4). When enabling *SCC*, Sherman's hierarchical lock [41] is removed to show the advantage of the one-sided SX lock. Note that DEFT can also be enhanced with the hierarchical lock to reduce the request frequency for remote locks. ***Leaf*** and ***Internal*** are the designs of the hash-based leaf node and the segmented internal node, respectively (§4.2 and §4.3).

On-path decoupled versioning (OPDV). As depicted in Figure 9(a), when the index cache is adequate, OPDV brings 46% improvement over the Baseline. ODPV removes the requirement for checksum calculation and enables fine-grained update or insert operations in tree nodes. The effect in the other case (cache disabled) is not obvious, because the system is bottlenecked by the network bandwidth due to the need for multiple round trips during tree search.

Scalable concurrency coordination. The shared-exclusive lock and the enabled concurrent upsert scheme boost the throughput by 2.5× and reduce the latency by 60% when the index cache is adequate. When disabling the index cache, though we have applied the hash-based leaf node and the segmented internal node first to reduce the I/O amplification, the system is still bottlenecked by the network bandwidth due to the need for multiple round trips during tree search. Therefore, the improvement is about 18%.

Hash-based leaf node. In the current implementation, the hash-based leaf node reduces the read granularity from an entire leaf node (1 KB) to a hash bucket pair (128 bytes). When the index cache is adequate, the leaf nodes take up the majority of remote accesses. Thus the hash-based leaf node improves the throughput by 2.1× and achieves 53% latency reduction. When the index cache is disabled, the improvement is about 5% because the majority of remote accesses are occupied by internal nodes.

Segmented internal node. The effect of the segmented internal node is opposite to that of the hash-based leaf node in terms of the index cache. It brings about a 56% improvement in throughput and a 36% reduction in latency when the index cache is disabled, and helps little when nearly not accessing internal nodes remotely.

Putting them together, these techniques reduce I/O amplification and improve concurrency performance under different situations. Specifically, DEFT improves the system throughput by 7.9× and 2.1× when the index cache is enabled and disabled, and reduces the average latency by 87% and 52%, respectively.

5.4 Sensitivity Analysis

In this section, we evaluate DEFT with different workload characteristics. The settings other than the adjustable ones are kept the same as in §5.2.

Uniform workloads. We first evaluate DEFT on the update-heavy (YCSB-A) and the read-only (YCSB-C) workloads with uniform distribution. Figure 10 shows the throughput and latency of these indexes. We have the following observations from the results. First, DEFT outperforms dLSM, Sherman-C, and SMART in terms of throughput by 9.5×, 2.3×, and 2.4× in the YCSB-A workload, and 46.3×, 4.9×, and 5.9× under the YCSB-C workload, respectively. Second, Sherman-C gets higher performance under the YCSB-A workload with uniform distribution than with skewed distribution due to the low contention on remote locks. Nevertheless, Sherman-C still does not scale well when the number of threads is high, because it is bottlenecked by the network bandwidth. Third, dLSM and SMART have even poorer performance under uniform workloads than under skewed workloads. Under skewed workloads, dLSM can gather hot items in MemTables

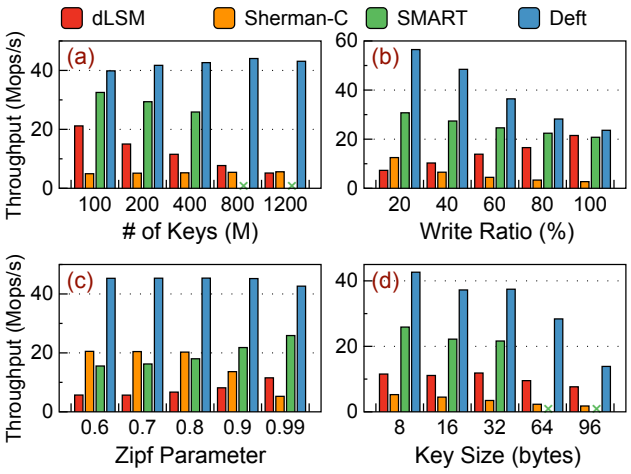

Figure 11. Sensitivity Analysis. *(a) Impact of the number of keys. (b) Impact of the read/write ratio. (c) Impact of the skewness. (d) Impact of the key size.*

in clients and get them without remote access, however, this is less likely under uniform distribution. For SMART, due to its large index size, the size-limited index cache has a lower hit ratio under uniform workloads.

Number of keys. Figure 11(a) shows the performance of indexes with different numbers of keys. We evaluate these indexes with 300 client threads on 10 compute nodes (and the same below). With the number of keys increasing, the throughput of Deft increases a little, this is because RDMA atomic operations have limited scalability, and larger key space can alleviate the contention slightly. Similarly, Sherman-C's performance increases slightly due to the less contention on exclusive locks under the skewed distribution. SMART is more sensitive to dataset size. With larger datasets, SMART has a larger index size and thus lower performance, and is even not able to run with 800 million and more keys due to out-of-memory in the MNs. The performance of dLSM drops sharply with larger datasets. This is because larger datasets have more SSTables and compaction operations, which reduces both the read and write performance.

Read/Write ratio. Figure 11(b) shows the results of indexes with different read/write ratios. With more write operations, dLSM has higher performance due to its write batching scheme. The throughput of other indexes drops with more write operations since write operations are more expensive than read operations. Sherman-C suffers more (about 78.4% from 20% write to 100% write) due to the more intensive contention on exclusive locks.

Skewness. Figure 11(c) shows the impact of skewness on these indexes. Deft performs best and is not sensitive to skewness owing to the scalable concurrency coordination. dLSM and SMART have better performance with higher skewness. For dLSM, the tiering structure of the LSM-tree can take advantage of the skewness, keeping hot items in MemTables or upper levels of SSTables. SMART has a higher index cache hit ratio with higher skewness. Sherman-C's throughput drops by 74% when the skewness increases from 0.6 to 0.99 due to the intense contention on exclusive locks.

Key size. Then we evaluate how the key size affects the performance of these indexes. For Deft and Sherman-C, we fix the number of entries in tree nodes and expand the size of the tree node accordingly to keep the tree height and avoid intrusive modification on the node structure. Figure 11(d) shows the results of these indexes with different key sizes. With the key size increasing, the throughput of Deft, Sherman-C, and SMART drops. Two main reasons account for the throughput decline of Deft and Sherman-C. First, with the node size of Deft and Sherman-C growing, the I/O amplification increases, and their performance is bottlenecked by bandwidth when the key size is large. Second, the throughput of both Deft and Sherman-C drops significantly when the key is larger than 64 bytes, which is because the index cache is not sufficient for the larger tree size. However, Deft still outperforms Sherman-C by 7.8-12.3× due to Deft's finer access granularity of segmented internal nodes and hash-based leaf nodes. For SMART, similar to larger key numbers, larger key sizes increase the index size and the size of SSTables significantly, so it cannot run when the key size is larger than 64 bytes due to out-of-memory in the MNs. dLSM is less affected by the key size and even has higher throughput with larger key sizes. This is because dLSM has fixed block-sized read granularity, and the read amplification of a single `RDMA_READ` is reduced with larger key sizes. Besides, the multiple MemTables with skewed distribution can help to absorb both remote reads and writes to some extent.

5.5 End-to-end Performance

In this experiment, we evaluate key-value store performance with Deft and other compared indexes using three representative workloads from Twitter [48]. The characteristics of these workloads are shown in Table 2. For Deft and Sherman, they store KV objects out of the index and require one additional RDMA operation to read or write each object. By contrast, dLSM and SMART can store KV objects inside the index (i.e., SSTables in dLSM and leaf nodes in SMART), so they do not need the additional RDMA operation for each object. Note that SMART still needs to store the KV objects out of the index for variable-sized values [25]. We evaluate these indexes with 4 MNs and 10 CNs. Except for dLSM which uses 300 client threads, other indexes use 900 client threads with coroutines. Other settings are the same as in §5.2.

The results are shown in Figure 12. For *Storage* workload that has a large object size, Deft outperforms Sherman and Sherman-C by 20.2% and 51.4%, respectively. dLSM and SMART cannot run the workload due to out-of-memory. The advantage of Deft is overshadowed by large values since the large value accelerates the network bandwidth saturation. Fortunately, recent real-world key-value workload analysis

Category	Object Size	Get Ratio
Storage	835 B	86%
Computation	215 B	50%
Transient	81 B	96%

Table 2. Twitter workloads description.

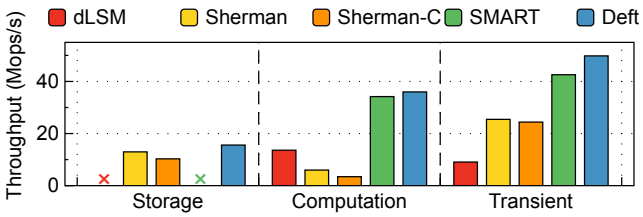

Figure 12. End-to-end performance.

shows that object sizes are typically small [8, 48]. *Computation* and *Transient* workloads have small object sizes that are much smaller than a typical index node size (1 KB). Thus, DEFT has great improvements over Sherman and Sherman-C by 2.0-10.4× owing to small I/O amplification. Similarly, SMART also achieves good performance due to small I/O amplification. Note that SMART embeds KV objects in leaf nodes and saves one round-trip for each object. Nevertheless, SMART still requires more round-trips to search its index, and thus is outperformed by DEFT by 5-17%.

6 Related Work

We discuss two related topics in this section: memory disaggregation and RDMA-based indexes.

Memory Disaggregation. Memory disaggregation has attracted widespread interest from both academia and industry in recent years. The industry has made great efforts in DM on both hardware and systems. HPE's "The Machine" [2, 18] develops a new type of computer architecture that connects many individual nodes over a memory fabric. IBM's ThymesisFlow [33] proposes a hardware-software co-designed memory disaggregation architecture that is interconnected by the OpenCAPI port. AliCloud's PolarDB-Serverless [7] introduces a novel cloud-native database that decouples the CPU resource from the memory pool.

Recently, there has been a great deal of research work exploring how to design efficient systems on DM. Clio [15] is a hardware-software co-designed DM system that includes OS functionalities and a framework for computation offloading. Clover [40] explores how to manage disaggregated persistent memory with a separate management strategy of the data plane and the metadata plane. FORD [51] proposes a one-sided distributed transaction system for disaggregated persistent memory. FUSEE [35] proposes a KV store on DM that supports a client-centric replication protocol. DEFT focuses on building an efficient index structure, the indispensable component of data management systems, on DM.

RDMA-based Indexes. Previous research explored efficient RDMA-based indexes on distributed memory systems [13, 16, 27, 28, 45]. However, these works are designed for traditional servers that rely on computing resources on remote servers.

Besides dLSM [43], Sherman [41], and SMART [25] mentioned in §3 and §5, there are a few works that propose RDMA-based indexes on DM. FG [57] is the first B$^+$Tree for DM with one-sided write operations. RACE hashing [58] is the first hashing index designed for DM. ROLEX [24] builds a learned index on the client side and manages data on DM via one-sided operations. ROLEX also suffers from deficiencies described in §3, since it will read multiple leaf nodes to tolerate prediction errors of learned models in a search operation, and uses exclusive locks for write operations. ROMe-KV adopts an append-only log update mode to guarantee wait-free consistent reads over RDMA [29]. Different from these works, DEFT is a tailored tree index for DM with small I/O amplification and high concurrency performance.

7 Conclusion

We propose DEFT, a scalable RDMA-based tree index on disaggregated memory. DEFT achieves low I/O amplification by designing fine-grained access patterns of node structures, and scales well with a scalable write-write concurrency coordination scheme on top of the tailored node structure and a lightweight read-write concurrency control technique for optimistic search. Evaluation results demonstrate that DEFT provides scalable performance and outperforms state-of-the-art tree indexes on disaggregated memory under various workloads. The prototype of DEFT is available at https://github.com/thustorage/deft.

Acknowledgments

We sincerely thank our shepherd Aishwarya Ganesan and the anonymous reviewers for their valuable comments and suggestions. This work is supported by the National Natural Science Foundation of China (U22B2023), Young Elite Scientists Sponsorship Program by CAST (2023QNRC001), and Postdoctoral Fellowship Program of CPSF (GZC20231296).

References

[1] 2023. Compute Express Link: The Breakthrough CPU-to-Device Interconnect. https://www.computeexpresslink.org/.
[2] 2023. The Machine: A new kind of computer. https://www.hpl.hp.com/research/systems-research/themachine/.
[3] 2023. VMware Research: Remote memory. https://research.vmware.com/projects/remote-memory.
[4] InfiniBand Trade Association et al. 2015. *InfiniBandTM Architecture Specification Volume 1 Release 1.3 (General Specifications).*
[5] Amanda Baran, Jacob Nelson-Slivon, Lewis Tseng, and Roberto Palmieri. 2024. ALock: Asymmetric Lock Primitive for RDMA Systems. In *Proceedings of the 36th ACM Symposium on Parallelism in*

Algorithms and Architectures (Nantes, France) *(SPAA '24)*. Association for Computing Machinery, New York, NY, USA, 15–26. https://doi.org/10.1145/3626183.3659977

[6] Tony Brewer and Nathan Kalyanasundharam. 2022. CXL3 Fabric introduction & use cases. In *Proc. Hot Chips*.

[7] Wei Cao, Yingqiang Zhang, Xinjun Yang, Feifei Li, Sheng Wang, Qingda Hu, Xuntao Cheng, Zongzhi Chen, Zhenjun Liu, Jing Fang, Bo Wang, Yuhui Wang, Haiqing Sun, Ze Yang, Zhushi Cheng, Sen Chen, Jian Wu, Wei Hu, Jianwei Zhao, Yusong Gao, Songlu Cai, Yunyang Zhang, and Jiawang Tong. 2021. PolarDB Serverless: A Cloud Native Database for Disaggregated Data Centers. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2477–2489. https://doi.org/10.1145/3448016.3457560

[8] Zhichao Cao, Siying Dong, Sagar Vemuri, and David H.C. Du. 2020. Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook. In *18th USENIX Conference on File and Storage Technologies (FAST 20)*. USENIX Association, Santa Clara, CA, 209–223. https://www.usenix.org/conference/fast20/presentation/cao-zhichao

[9] Hokeun Cha, Xiangpeng Hao, Tianzheng Wang, Huanchen Zhang, Aditya Akella, and Xiangyao Yu. 2023. Blink-hash: An Adaptive Hybrid Index for In-Memory Time-Series Databases. *Proc. VLDB Endow.* 16, 6 (feb 2023), 1235–1248. https://doi.org/10.14778/3583140.3583143

[10] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: A Concurrent Key-Value Store with In-Place Updates. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 275–290. https://doi.org/10.1145/3183713.3196898

[11] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing* (Indianapolis, Indiana, USA) *(SoCC '10)*. Association for Computing Machinery, New York, NY, USA, 143–154. https://doi.org/10.1145/1807128.1807152

[12] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 215–226. https://doi.org/10.1145/2882903.2903741

[13] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. 2014. FaRM: Fast Remote Memory. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. USENIX Association, Seattle, WA, 401–414. https://www.usenix.org/conference/nsdi14/technical-sessions/dragojevi%C4%87

[14] Tomasz Gromadzki and Jan Marian Michalski. 2019. Persistent Memory Replication Over Traditional RDMA Part 1: Understanding Remote Persistent Memory. https://www.intel.com/content/www/us/en/developer/articles/technical/persistent-memory-replication-over-traditional-rdma-part-1-understanding-remote-persistent.html.

[15] Zhiyuan Guo, Yizhou Shan, Xuhao Luo, Yutong Huang, and Yiying Zhang. 2022. Clio: A Hardware-Software Co-Designed Disaggregated Memory System. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 417–433. https://doi.org/10.1145/3503222.3507762

[16] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2014. Using RDMA Efficiently for Key-Value Services. *SIGCOMM Comput. Commun. Rev.* 44, 4 (aug 2014), 295–306. https://doi.org/10.1145/2740070.2626299

[17] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. Design Guidelines for High Performance RDMA Systems. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*. USENIX Association, Denver, CO, 437–450. https://www.usenix.org/conference/atc16/technical-sessions/presentation/kalia

[18] Kimberly Keeton. 2017. Memory-Driven Computing. USENIX Association, Santa Clara, CA.

[19] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 489–504. https://doi.org/10.1145/3183713.3196909

[20] Leslie Lamport. 1974. A New Solution of Dijkstra's Concurrent Programming Problem. *Commun. ACM* 17, 8 (aug 1974), 453–455. https://doi.org/10.1145/361082.361093

[21] Sekwon Lee, Soujanya Ponnapalli, Sharad Singhal, Marcos K. Aguilera, Kimberly Keeton, and Vijay Chidambaram. 2022. DINOMO: An Elastic, Scalable, High-Performance Key-Value Store for Disaggregated Persistent Memory. *Proc. VLDB Endow.* 15, 13 (sep 2022), 4023–4037. https://doi.org/10.14778/3565838.3565854

[22] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. 2013. The Bw-Tree: A B-tree for new hardware platforms. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 302–313. https://doi.org/10.1109/ICDE.2013.6544834

[23] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 574–587. https://doi.org/10.1145/3575693.3578835

[24] Pengfei Li, Yu Hua, Pengfei Zuo, Zhangyu Chen, and Jiajie Sheng. 2023. ROLEX: A Scalable RDMA-oriented Learned Key-Value Store for Disaggregated Memory Systems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. USENIX Association, Santa Clara, CA, 99–114. https://www.usenix.org/conference/fast23/presentation/li-pengfei

[25] Xuchuan Luo, Pengfei Zuo, Jiacheng Shen, Jiazhen Gu, Xin Wang, Michael R. Lyu, and Yangfan Zhou. 2023. SMART: A High-Performance Adaptive Radix Tree for Disaggregated Memory. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 553–571. https://www.usenix.org/conference/osdi23/presentation/luo

[26] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 742–755. https://doi.org/10.1145/3582016.3582063

[27] Christopher Mitchell, Yifeng Geng, and Jinyang Li. 2013. Using One-Sided RDMA Reads to Build a Fast, CPU-Efficient Key-Value Store. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*. USENIX Association, San Jose, CA, 103–114. https://www.usenix.org/conference/atc13/technical-sessions/presentation/mitchell

[28] Christopher Mitchell, Kate Montgomery, Lamont Nelson, Siddhartha Sen, and Jinyang Li. 2016. Balancing CPU and Network in the Cell Distributed B-Tree Store. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*. USENIX Association, Denver, CO, 451–464. https://www.usenix.org/conference/atc16/technical-sessions/presentation/mitchell

[29] Jacob Nelson-Slivon, Reilly Yankovich, Ahmed Hassan, and Roberto Palmieri. 2024. Brief Announcement: ROMe: Wait-free Objects for RDMA. In *Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures* (Nantes, France) *(SPAA '24)*. Association for Computing Machinery, New York, NY, USA, 371–373. https://doi.org/10.1145/3626183.3660262

[30] Vlad Nitu, Boris Teabe, Alain Tchana, Canturk Isci, and Daniel Hagimont. 2018. Welcome to Zombieland: Practical and Energy-Efficient Memory Disaggregation in a Datacenter. In *Proceedings of the Thirteenth EuroSys Conference* (Porto, Portugal) *(EuroSys '18)*. Association for Computing Machinery, New York, NY, USA, Article 16, 12 pages. https://doi.org/10.1145/3190508.3190537

[31] NVIDIA. 2023. *NVIDIA MLNX_OFED Documentation.*

[32] PCI-SIG. 2014. *PCI Express Base Specification Revision 4.0.*

[33] Christian Pinto, Dimitris Syrivelis, Michele Gazzetti, Panos Koutsovasilis, Andrea Reale, Kostas Katrinis, and H. Peter Hofstee. 2020. ThymesisFlow: A Software-Defined, HW/SW co-Designed Interconnect Stack for Rack-Scale Memory Disaggregation. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 868–880. https://doi.org/10.1109/MICRO50266.2020.00075

[34] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 69–87. https://www.usenix.org/conference/osdi18/presentation/shan

[35] Jiacheng Shen, Pengfei Zuo, Xuchuan Luo, Tianyi Yang, Yuxin Su, Yangfan Zhou, and Michael R. Lyu. 2023. FUSEE: A Fully Memory-Disaggregated Key-Value Store. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. USENIX Association, Santa Clara, CA, 81–98. https://www.usenix.org/conference/fast23/presentation/shen

[36] IBM Research Editorial Staff. 2023. Advancing cloud with memory disaggregation. https://www.ibm.com/blogs/research/2018/01/advancing-cloud-memory-disaggregation/.

[37] Yupeng Tang, Ping Zhou, Wenhui Zhang, Henry Hu, Qirui Yang, Hao Xiang, Tongping Liu, Jiaxin Shan, Ruoyun Huang, Cheng Zhao, Cheng Chen, Hui Zhang, Fei Liu, Shuai Zhang, Xiaoning Ding, and Jianjun Chen. 2024. Exploring Performance and Cost Optimization with ASIC-Based CXL Memory. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 818–833. https://doi.org/10.1145/3627703.3650061

[38] Mellanox Technologies. 2020. *Mellanox Adapters Programmer's Reference Manual (PRM) v0.52.*

[39] Mellanox Technologies. 2020. *RDMA Aware Networks Programming User Manual v1.7.*

[40] Shin-Yeh Tsai, Yizhou Shan, and Yiying Zhang. 2020. Disaggregating Persistent Memory and Controlling Them Remotely: An Exploration of Passive Disaggregated Key-Value Stores. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 33–48. https://www.usenix.org/conference/atc20/presentation/tsai

[41] Qing Wang, Youyou Lu, and Jiwu Shu. 2022. Sherman: A Write-Optimized Distributed B+Tree Index on Disaggregated Memory. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 1033–1048. https://doi.org/10.1145/3514221.3517824

[42] Ruihong Wang, Jianguo Wang, Stratos Idreos, M. Tamer Özsu, and Walid G. Aref. 2022. The Case for Distributed Shared-Memory Databases with RDMA-Enabled Memory Disaggregation. *Proc. VLDB Endow.* 16, 1 (sep 2022), 15–22. https://doi.org/10.14778/3561261.3561263

[43] Ruihong Wang, Jianguo Wang, Prishita Kadam, M. Tamer Özsu, and Walid G. Aref. 2023. dLSM: An LSM-Based Index for Memory Disaggregation. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*.

[44] Ziqi Wang, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G. Andersen. 2018. Building a Bw-Tree Takes More Than Just Buzz Words. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 473–488. https://doi.org/10.1145/3183713.3196895

[45] Xingda Wei, Rong Chen, and Haibo Chen. 2020. Fast RDMA-based Ordered Key-Value Store using Remote Learned Cache. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 117–135. https://www.usenix.org/conference/osdi20/presentation/wei

[46] Xingda Wei, Jiaxin Shi, Yanzhe Chen, Rong Chen, and Haibo Chen. 2015. Fast in-memory transaction processing using RDMA and HTM. In *Proceedings of the 25th Symposium on Operating Systems Principles* (Monterey, California) *(SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 87–104. https://doi.org/10.1145/2815400.2815419

[47] Xingda Wei, Haotian Wang, Tianxia Wang, Rong Chen, Jinyu Gu, Pengfei Zuo, and Haibo Chen. 2023. Transactional Indexes on (RDMA or CXL-based) Disaggregated Memory with Repairable Transaction. arXiv:2308.02501 [cs.DB]

[48] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2020. A large scale analysis of hundreds of in-memory cache clusters at Twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 191–208. https://www.usenix.org/conference/osdi20/presentation/yang

[49] Dong Young Yoon, Mosharaf Chowdhury, and Barzan Mozafari. 2018. Distributed Lock Management with RDMA: Decentralization without Starvation. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 1571–1586. https://doi.org/10.1145/3183713.3196890

[50] Bowen Zhang, Shengan Zheng, Zhenlin Qi, and Linpeng Huang. 2022. NBTree: A Lock-Free PM-Friendly Persistent B+-Tree for EADR-Enabled PM Systems. *Proc. VLDB Endow.* 15, 6 (feb 2022), 1187–1200. https://doi.org/10.14778/3514061.3514066

[51] Ming Zhang, Yu Hua, Pengfei Zuo, and Lurong Liu. 2022. FORD: Fast One-sided RDMA-based Distributed Transactions for Disaggregated Persistent Memory. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. USENIX Association, Santa Clara, CA, 51–68. https://www.usenix.org/conference/fast22/presentation/zhang-ming

[52] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 658–674. https://doi.org/10.1145/3600006.3613135

[53] Qizhen Zhang, Yifan Cai, Xinyi Chen, Sebastian Angel, Ang Chen, Vincent Liu, and Boon Thau Loo. 2020. Understanding the Effect of Data Center Resource Disaggregation on Production DBMSs. *Proc. VLDB Endow.* 13, 9 (may 2020), 1568–1581. https://doi.org/10.14778/3397230.3397249

[54] Qizhen Zhang, Xinyi Chen, Sidharth Sankhe, Zhilei Zheng, Ke Zhong, Sebastian Angel, Ang Chen, Vincent Liu, and Boon Thau Loo. 2022. Optimizing Data-Intensive Systems in Disaggregated Data Centers with TELEPORT. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 1345–1359. https://doi.org/10.1145/3514221.3517856

[55] Yingqiang Zhang, Chaoyi Ruan, Cheng Li, Xinjun Yang, Wei Cao, Feifei Li, Bo Wang, Jing Fang, Yuhui Wang, Jingze Huo, and Chao Bi. 2021. Towards Cost-Effective and Elastic Cloud Database Deployment via Memory Disaggregation. *Proc. VLDB Endow.* 14, 10 (jun 2021),

1900–1912. https://doi.org/10.14778/3467861.3467877

[56] Tobias Ziegler, Jacob Nelson-Slivon, Viktor Leis, and Carsten Binnig. 2023. Design Guidelines for Correct, Efficient, and Scalable Synchronization Using One-Sided RDMA. *Proc. ACM Manag. Data* 1, 2, Article 131 (jun 2023), 26 pages. https://doi.org/10.1145/3589276

[57] Tobias Ziegler, Sumukha Tumkur Vani, Carsten Binnig, Rodrigo Fonseca, and Tim Kraska. 2019. Designing Distributed Tree-Based Index Structures for Fast RDMA-Capable Networks. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 741–758. https://doi.org/10.1145/3299869.3300081

[58] Pengfei Zuo, Jiazhao Sun, Liu Yang, Shuangwu Zhang, and Yu Hua. 2021. One-sided RDMA-Conscious Extendible Hashing for Disaggregated Memory. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 15–29. https://www.usenix.org/conference/atc21/presentation/zuo